Object-Oriented Programming in Java

Java Exercises

Exercises

28 hands-on coding exercises on the five slide decks J01–J04: the Java environment, Java basics, characters and strings, JUnit 5, inheritance and interfaces. Difficulty rises inside each part, from ★ warm-up to ★★★ challenge. Most exercises end with a `main` or a test class and the sample run your program must print.

Contents

How to use this sheet **2**

Part 1 – The Java environment (J01) **2**
1.1 ★ Hello, command line . . . 2
1.2 ★ Fix the conventions . . . 3
1.3 ★★ Jar file and magic number . . . 3

Part 2 – Java basics (J02) **3**
2.1 ★ Calendar utilities . . . 3
2.2 ★ Counter and Car . . . 4
2.3 ★★ Encapsulated bank account . . . 5
2.4 ★★ A shop in two packages . . . 5
2.5 ★★ Number tools: wrappers, Scanner, casts . . . 6
2.6 ★★ Array tools . . . 7
2.7 ★★ static, final, factory methods, singleton . . . 7
2.8 ★★ Cards: enum and record . . . 8
2.9 ★★★ Nested classes . . . 9

Part 3 – Characters and strings (J02a) **9**
3.1 ★ Text utilities . . . 9
3.2 ★★ A formatted receipt . . . 10
3.3 ★★ StringBuilder, speed and the string pool . . . 11
3.4 ★★★ Encoding tools . . . 12

Part 4 – JUnit 5 (J03) **12**
4.1 ★ Test the slides' Stack . . . 12
4.2 ★★ Test the bank account, then a suite . . . 13
4.3 ★★★ Bug reproduction and TDD . . . 13

Part 5 – Inheritance, interfaces, lambdas (J04) **14**
5.1 ★ Employees, managers, CEO . . . 14
5.2 ★★ equals, hashCode, toString, clone . . . 15
5.3 ★★ Abstract classes and Template Method . . . 15
5.4 ★★ Interfaces: complex numbers, default methods . . . 16
5.5 ★★★ Composite expression tree . . . 17
5.6 ★★★ Comparable, Comparator, Strategy . . . 18
5.7 ★★★ Iterable and Iterator . . . 18
5.8 ★★★ Functional interfaces, lambdas, method references . . . 19
5.9 ★★★ Observer by composition (code + test) . . . 20

How to use this sheet

28 coding exercises cover the topics of the five slide decks, in deck order, and get harder inside each part: ★ warm-up, ★★ core, ★★★ challenge. Every exercise asks you to write Java code; most end with a `main` (or a test class) and a **sample run** to check your program against. Solutions are in the companion PDF, *Java Exercises – Solutions*.

Use Java 25 (the slides use `IO.println` and `void main()`); Part 4 needs JUnit 5. One package per exercise keeps names apart, e.g. `ex2_3` for exercise 2.3.

Where an exercise gives code followed by a sample run, that code is the body of `main`. When it calls static methods without a class name (2.5, 2.6, 3.1–3.4), put that `main` inside the class you write; in 5.2, 5.5 and 5.8, put the static methods the exercise asks for (`countDistinct`, `show`, `forEachIf`, ...) in the same class as `main`; otherwise use a separate class (in 2.7, not inside `IdGenerator`). Part 1 uses the default package (no `package` line).

Part	Slide deck	Exercises	Topics
1	J01 Java Environment	1.1–1.3	javac/java, command-line arguments, Java 25 `void main()` + `IO`, coding conventions, jar + manifest, 0xCAFEBABE
2	J02 Java Basics	2.1–2.9	control statements, switch on strings, boolean operators, classes, constructors, overloading, `this`, method chaining, aliasing vs copy, static members, encapsulation, final, constants, packages and visibility, wrappers, autoboxing, Scanner, casts, arrays, jagged arrays, varargs, `Arrays`, utility classes, static import, factory methods, singleton, static blocks, immutability, enum, record, static nested / inner / local classes
3	J02a Chars and Strings	3.1–3.4	`Character`, String methods, `compareTo`, format/formatted, StringBuilder/StringBuffer and their speed, string pool and `intern()`, `Charset`, UTF-8 / UTF-16 / ISO-8859-1, mojibake, code points
4	J03 JUnit	4.1–4.3	@Test, assertions with messages and tolerance, assertThrows, @BeforeEach/@AfterEach/@BeforeAll/@AfterAll, suites, @Disabled, assumptions, TDD, bug reproduction
5	J04 Inheritance	5.1–5.9	extends, override, super, polymorphism, dynamic binding, casts, instanceof, protected, final methods, toString/equals/hashCode, Cloneable, abstract classes, Template Method, interfaces, static/default methods, anonymous classes, Composite, flag interfaces, Comparable/Comparator, Strategy, Iterable/Iterator, functional interfaces, lambdas, method references, Observer, composition

Part 1 – The Java environment (J01)

1.1 ★ Hello, command line

Write `Greeter.java`: for each command-line argument it prints `Hello, <arg>!` on its own line. With no arguments it asks for a name with `IO.readln("Name? ")` and greets it, or greets `world` when the user just presses Enter. Compile it with `javac` and run it from the terminal. Then write `GreeterCompact.java`, the same program in the Java 25 form from the slides (no `public static`).

Sample run:

```
$ java Greeter Ada Linus
Hello, Ada!
Hello, Linus!
$ java Greeter
Name? Gabriele
Hello, Gabriele!
$ java Greeter
Name?
Hello, world!
```

1.2 ★ Fix the conventions

Rename everything in this class so it follows the course's coding conventions (same behaviour), and save it in the right file.

```java
public class bank_account {
    static final double interestrate = 0.02;
    private double Balance;
    public void Deposit_Money(double AMOUNT) { Balance += AMOUNT; }
    public double get_balance() { return Balance; }
}
```

1.3 ★★ Jar file and magic number

1. Package `Greeter` into `greeter.jar` with a manifest, so that `java -jar greeter.jar Ada` prints `Hello, Ada!`. Also list the jar's contents, and run `Greeter` from the jar through the classpath, without the manifest.
2. Write `MagicNumber.java`: it reads the file named by `args[0]`, prints its first 4 bytes in hexadecimal, and says whether it is a Java class file (class files start with `CAFEBABE`). Read the file with `byte[] data = java.nio.file.Files.readAllBytes(java.nio.file.Path.of(args[0]));` and declare `main(...) throws Exception`. Careful: Java bytes are signed.

Sample run (the `.java` line shows the codes of its first four characters, `publ`):

```
$ java MagicNumber Greeter.class
Greeter.class starts with CAFEBABE
It is a Java class file
$ java MagicNumber Greeter.java
Greeter.java starts with 7075626C
It is not a Java class file
```

Part 2 – Java basics (J02)

2.1 ★ Calendar utilities

Write a class `CalendarUtils` with these static methods:

- `String season(String month)`: a switch on strings with grouped cases – december–february → `"winter"`, march–may → `"spring"`, june–august → `"summer"`, september–november → `"autumn"`, anything else → `"unknown"`. Case must not matter.

- `boolean isLeap(int year)`: divisible by 4 and not by 100, or divisible by 400 – one boolean expression, no `if`.
- `int daysInMonth(String month, int year)`: 28/29/30/31, or -1 for an unknown month.
- `int daysInYear(int year)`: a loop over an array of the 12 month names.
- `int nextLeapYear(int year)`: the first leap year after `year`, found with a `while` loop.
- `main`: for each year on the command line (converted with `Integer.parseInt`; 2023, 2024, 1900, 2000 when there are none), print its number of days and mark leap years; then print the next leap year after 2025 and the seasons of `"JULY"` and `"Smarch"`.

Sample run, no arguments:

```
2023: 365 days
2024: 366 days (leap)
1900: 365 days
2000: 366 days (leap)
Next leap year after 2025: 2028
JULY -> summer, Smarch -> unknown
```

2.2 ★ Counter and Car

1. Complete the slides' `Counter` (they call `decrement(7)` but never define it): a private `int value`; constructors `Counter()` and `Counter(int start)`, the first delegating with `this(...)` (written inside a constructor, `this(args)` calls another constructor of the same class); `reset()`, `increment(int by)`, `decrement(int by)` and `print()` (prints the value with `System.out.println`), all returning `this`; a query `getValue()`.
2. Write `Car` with private `brand`, `color`, `turnedOn` and:
   - constructors `Car()` (a red Ferrari), `Car(String brand)` (white), `Car(String brand, String color)`, and a copy constructor `Car(Car other)` that builds an independent copy; all of them go through the two-argument one with `this(...)`;
   - a private static counter of the cars built, with a static getter `getCountBuiltCars()`;
   - `turnOn()`, `paint(String color)` (use `this` to tell the attribute from the parameter) and an overloaded `paint()` that paints it white, `isOn()`, getters, `sameAs(Car other)` comparing brand and color, and `toString()` like `Ferrari red (off)`.

Run this `main`:

```
new Counter(3).print().reset().print().increment(10).print().decrement(7).print();

Car a = new Car();
Car b = a;
Car c = new Car(a);
b.paint("yellow");
c.turnOn();
IO.println(a + " | " + b + " | " + c);
IO.println((a == b) + " " + (a == c) + " " + a.sameAs(new Car("Ferrari", "yellow")));
IO.println(new Car("Fiat") + ", " + new Car("Lancia", "blue"));
IO.println("Cars built: " + Car.getCountBuiltCars());
```

Sample run:

```
3
0
10
3
Ferrari yellow (off) | Ferrari yellow (off) | Ferrari red (on)
true false true
Fiat white (off), Lancia blue (off)
Cars built: 5
```

2.3 ★★ Encapsulated bank account

Write `BankAccount`:

- a public constant `MAX_WITHDRAWAL = 1000.0`;
- a `number` assigned automatically (1, 2, 3, … from a private static counter), an `owner` and a `balance`, all private; number and owner never change (`final`);
- constructors `BankAccount(String owner)` (balance 0) and `BankAccount(String owner, double initialBalance)`; a negative initial balance throws `new IllegalArgumentException("...")` and does not use up a number;
- `deposit(double amount)` (amount > 0) and `withdraw(double amount)` (amount > 0, $\leq$ `MAX_WITHDRAWAL`, $\leq$ balance); invalid amounts throw `IllegalArgumentException`;
- getters, a static `getCreatedAccounts()`, and `toString()` like `#1 Alice: 120.0`. No `setBalance`.

Run this `main` (you will write proper tests in 4.2):

```
BankAccount a = new BankAccount("Alice", 100);
BankAccount b = new BankAccount("Bob");
a.deposit(50);
a.withdraw(30);
b.deposit(BankAccount.MAX_WITHDRAWAL);
b.withdraw(BankAccount.MAX_WITHDRAWAL);
IO.println(a);
IO.println(b);
IO.println("Accounts: " + BankAccount.getCreatedAccounts());
```

Sample run:

```
#1 Alice: 120.0
#2 Bob: 0.0
Accounts: 2
```

2.4 ★★ A shop in two packages

Build this small project:

- `it.polito.shop.Product` (public): private final `name` and `price`, a package-visible `int stock`, a package-visible constructor, public getters, and a package-visible `removeStock(int qty)`.
- `it.polito.shop.Discounts` (**not** public): `static double apply(double price, int qty)` returns price × qty, with 10% off when qty ≥ 10.
- `it.polito.shop.Shop` (public): products in an array whose capacity is given to the constructor; `addProduct(name, price, stock)` returns false when full or when the name already exists; `find(name)` returns the `Product` or `null`; `sell(name, qty)` lowers the stock and returns the

amount paid (using `Discounts`), or -1 when the product is unknown, qty ≤ 0 or the stock is too low; `stockOf(name)` (0 for an unknown product).
- `it.polito.client.ShopApp`: a `main` whose only import is `import it.polito.shop.*;`.

Compile from the project folder with `javac -d out $(find src -name "*.java")` and run `java -cp out it.polito.client.ShopApp`. Then check that `p.removeStock(1)`, `Discounts.apply(1.5, 10)` and `new Product("x", 1, 1)` written in `ShopApp` are all compile errors.

```
Shop shop = new Shop(10);
shop.addProduct("pen", 1.5, 20);
shop.addProduct("notebook", 4.0, 3);
IO.println(shop.sell("pen", 2));
IO.println(shop.sell("pen", 10));
IO.println(shop.sell("notebook", 5));
IO.println(shop.sell("eraser", 1));
IO.println(shop.stockOf("pen") + " " + shop.stockOf("notebook"));
Product p = shop.find("pen");
IO.println(p.getName() + " " + p.getPrice() + " " + p.getStock());
```

Sample run:

```
3.0
13.5
-1.0
-1.0
8 3
pen 1.5 8
```

2.5 ★★ Number tools: wrappers, Scanner, casts

Write `NumberTools`:

- `int sumInts(String line)`: a `Scanner` on the string sums every integer and skips the other tokens (`hasNext`, `hasNextInt`, `nextInt`, `next`).
- `String stats(String line)`: `count=3 min=10 max=30 avg=20.0`, or `count=0` when there are no integers. Start min/max from `Integer.MAX_VALUE`/`Integer.MIN_VALUE`; the average must not be truncated.
- `String bases(int n)`: `255 = 0b11111111 = 0xff = 0o377` (wrapper methods `Integer.toBinaryString`, `toHexString`, `toOctalString`).
- `int percent(int part, int total)`: the rounded percentage (1 of 3 → 33, 2 of 3 → 67).
- `boolean sameValue(Integer a, Integer b)`: true when the values are equal – it must work for every value, 1000 included.

```
IO.println("sumInts = " + sumInts("10 abc 30 4.5 -2"));
IO.println("stats   = " + stats("10 20 x 30"));
IO.println("stats   = " + stats("no numbers here"));
IO.println("bases   = " + bases(255));
IO.println("percent = " + percent(1, 3) + " " + percent(2, 3));
Integer x = 1000, y = 1000;
IO.println("sameValue = " + sameValue(x, y) + ", but x == y is " + (x == y));
```

Sample run:

```
sumInts = 38
stats   = count=3 min=10 max=30 avg=20.0
stats   = count=0
bases   = 255 = 0b11111111 = 0xff = 0o377
percent = 33 67
sameValue = true, but x == y is false
```

2.6 ★★ Array tools

Write `ArrayTools` (with `import java.util.Arrays;`):

- `int[][] pascal(int n)`: Pascal's triangle as a jagged array (row i has i+1 elements);
- `int[][] transpose(int[][] m)` for a rectangular matrix;
- `void swapRows(double[][] m, int i, int j)` in constant time, even when the rows have different lengths;
- `double average(double... values)` (NaN when called with no arguments);
- `int max(int first, int... others)`, which cannot be called with zero arguments;
- `int[] concatSorted(int[] a, int[] b)`: both arrays together, sorted (use `Arrays.copyOf` and `Arrays.sort`).

```
IO.println(Arrays.deepToString(pascal(5)));
IO.println(Arrays.deepToString(transpose(new int[][]{{1, 2, 3}, {4, 5, 6}})));
double[][] grid = {{1, 2}, {3, 4, 5}};
swapRows(grid, 0, 1);
IO.println(Arrays.deepToString(grid));
IO.println(average(1, 2, 3, 4) + " " + average(new double[]{2, 4}) + " " + average());
IO.println(max(4) + " " + max(9, 3, 15, 5));
int[] merged = concatSorted(new int[]{7, 2, 5}, new int[]{1, 8, 5});
IO.println(Arrays.toString(merged));
IO.println(Arrays.binarySearch(merged, 7));
int[] copy = Arrays.copyOf(merged, merged.length);
IO.println((copy == merged) + " " + Arrays.equals(copy, merged));
int[] filled = new int[4];
Arrays.fill(filled, -1);
IO.println(Arrays.toString(filled));
```

Sample run:

```
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
[[1, 4], [2, 5], [3, 6]]
[[3.0, 4.0, 5.0], [1.0, 2.0]]
2.5 3.0 NaN
4 15
[1, 2, 5, 5, 7, 8]
4
false true
[-1, -1, -1, -1]
```

2.7 ★★ static, final, factory methods, singleton

1. `Temperature`, an immutable class: private constructor; `public static final double ABSOLUTE_ZERO = -273.15`; static factories `ofCelsius(double)` (throws `IllegalArgumentException` below absolute zero, and returns the same pre-built object every time for 0) and `ofFahrenheit(double)`; queries `celsius()` and `fahrenheit()`; `plus(double`

`delta)` returning a new `Temperature`; `toString()` like `25.0 C`.

2. `IdGenerator`, a lazy singleton: private constructor that prints `IdGenerator created`; `getInstance()`; `nextId()` returning `ID-0001`, `ID-0002`, … (`String.format("%04d", ...)`). A `static final String PREFIX` is set in a static block, which also prints `IdGenerator class loaded`.
3. `Geometry`, a utility class (no instances): `distance(x1, y1, x2, y2)` and `circleArea(r)`, written with `import static java.lang.Math.*;` so that the method bodies call `sqrt` and `pow` and use `PI` without the `Math.` prefix.

```
Temperature t = Temperature.ofFahrenheit(212);
IO.println(t + " = " + t.fahrenheit() + " F");
IO.println(Temperature.ofCelsius(25).plus(-5));
IO.println((Temperature.ofCelsius(0) == Temperature.ofFahrenheit(32)) + " "
        + (Temperature.ofCelsius(20) == Temperature.ofCelsius(20)));
IO.println("start");
IdGenerator g = IdGenerator.getInstance();
IO.println(g.nextId() + " " + g.nextId());
IO.println(IdGenerator.getInstance().nextId() + " " + (g == IdGenerator.getInstance()));
IO.println(Geometry.distance(0, 0, 3, 4) + " " + Geometry.circleArea(1));
```

Sample run:

```
100.0 C = 212.0 F
20.0 C
true false
start
IdGenerator class loaded
IdGenerator created
ID-0001 ID-0002
ID-0003 true
5.0 3.141592653589793
```

2.8 ★★ Cards: enum and record

Write `enum Suit { SPADES, HEARTS, DIAMONDS, CLUBS }` and `record Card(int rank, Suit suit)` with:

- `label()`: `"A of SPADES"` for rank 1, J/Q/K for 11/12/13, the number otherwise (a switch);
- `withRank(int newRank)` returning a new `Card` (records are immutable);
- static `fullDeck()`: the 52 cards, looping over `Suit.values()` and ranks 1–13;
- static `deal(Card[] deck, int n)`: the first n cards (`Arrays.copyOfRange`);
- static `countBySuit(Card... cards)`: an `int[]` indexed by `suit().ordinal()`;
- static `highest(Card first, Card... others)`: the card with the highest rank.

```
Card[] deck = Card.fullDeck();
IO.println(deck.length + " cards: " + deck[0].label() + " ... " + deck[51].label());
Card[] hand = Card.deal(deck, 15);
IO.println(Arrays.toString(Card.countBySuit(hand)));
Card q = new Card(12, Suit.HEARTS);
IO.println(q + " -> " + q.label());
IO.println(q.equals(new Card(12, Suit.HEARTS)) + " " + q.withRank(1).label());
IO.println(Card.highest(new Card(3, Suit.CLUBS), q, new Card(7, Suit.SPADES)).label());
IO.println(Suit.valueOf("DIAMONDS").ordinal() + " " + Suit.CLUBS.name());
```

Sample run:

```
52 cards: A of SPADES ... K of CLUBS
[13, 2, 0, 0]
Card[rank=12, suit=HEARTS] -> Q of HEARTS
true A of HEARTS
Q of HEARTS
2 CLUBS
```

2.9 ★★★ Nested classes

1. `StackOfInt` as a linked list, with a private static nested class `Element { int value; Element next; }`: `push`, `pop`, `peek`, `isEmpty`, `size`, and `toString()` with the top first, e.g. `[3, 2, 1]`. `pop`/`peek` on an empty stack throw `new IllegalStateException("empty stack")`.
2. The slides' `Counter` with the inner class `Incrementer` (field `step`, constructor `Incrementer(int step)`, `doIncrement()` does `i += step`) and `buildIncrementer(int step)`.
3. Inside `main`, a local class `Multiplier` whose `apply(int x)` multiplies by a local variable `factor` of `main`.

```
StackOfInt st = new StackOfInt();
for (int i = 1; i <= 4; i++) {
    st.push(i);
}
IO.println(st + " size=" + st.size());
IO.println(st.pop() + " " + st.peek() + " " + st);

Counter c = new Counter();
Counter other = new Counter();
Counter.Incrementer byOne = c.buildIncrementer(1);
Counter.Incrementer byFour = c.buildIncrementer(4);
byOne.doIncrement();
byFour.doIncrement();
byFour.doIncrement();
other.buildIncrementer(100).doIncrement();
IO.println(c.getValue() + " " + other.getValue());

int factor = 3;
// your local class Multiplier here
Multiplier m = new Multiplier();
IO.println(m.apply(7) + " " + m.apply(-2));
```

Sample run:

```
[4, 3, 2, 1] size=4
4 3 [3, 2, 1]
9 100
21 -6
```

Part 3 – Characters and strings (J02a)

3.1 ★ Text utilities

Write `TextUtils` with static methods:

- `stats(String s)`: counts letters, digits, spaces and other characters with `Character.isLetter`, `isDigit`, `isSpaceChar`;

- `countOccurrences(String text, String word)`: non-overlapping occurrences, using `indexOf(String, int fromIndex)`;
- `capitalizeWords(String s)`: first letter of every word upper case, the rest lower case, spaces kept as they are;
- `initials(String fullName)`: the upper-case initials, ignoring extra spaces – `"Ada King Lovelace"` → `"A.K.L."`, `"  grace   hopper "` → `"G.H."` (`trim`, `indexOf`, `substring`, `charAt`);
- `extension(String fileName)`: the text after the last dot, `""` if there is none (`lastIndexOf`);
- `firstAlphabetical(String first, String... others)`: the alphabetically first word, ignoring case.

```
IO.println(stats("Java 25 rocks!"));
IO.println(countOccurrences("banana", "an") + " " + countOccurrences("aaaa", "aa") + " "
        + countOccurrences("abc", "d"));
IO.println(capitalizeWords("hello wORLD  java"));
IO.println(initials("Ada King Lovelace") + " " + initials("  grace   hopper "));
IO.println(extension("report.final.pdf") + "|" + extension("README") + "|");
IO.println(firstAlphabetical("pear", "Fig", "banana", "apple"));
```

Sample run:

```
letters=9 digits=2 spaces=2 others=1
2 2 0
Hello World  Java
A.K.L. G.H.
pdf||
apple
```

3.2 ★★ A formatted receipt

Write `record Item(int code, String name, int qty, double unitPrice)` with a `total()` method, and a class `Receipt`:

- `header()`: the column titles `CODE`, `ITEM`, `QTY`, `TOTAL`, built with `"...".formatted(...)`;
- `line(Item it)`: code zero-padded to 5 digits, a space, name left-aligned in 12, quantity right-aligned in 4, total with 2 decimals right-aligned in 9;
- `receipt(double discount, Item... items)`: the header, one line per item, a `Discount` line showing the amount in parentheses (the `(` flag), and a `TOTAL` line, each ending with `\n`;
- `footer(String cashier, int number)`: `Receipt #42, served by Gabriele`, using explicit argument indexes (`%2$d`, `%1$s`) so that the arguments stay in the order cashier, number.

Pass `Locale.US` to `String.format` for numbers, so the program prints decimal points on an Italian PC too.

```
IO.print(receipt(1.45,
        new Item(12, "Coffee", 2, 1.20),
        new Item(7, "Croissant", 3, 1.35),
        new Item(100, "Water", 10, 0.50)));
IO.println(footer("Gabriele", 42));
```

Sample run:

```
CODE  ITEM         QTY    TOTAL
00012 Coffee         2     2.40
00007 Croissant      3     4.05
00100 Water         10     5.00
      Discount           (1.45)
      TOTAL               10.00
Receipt #42, served by Gabriele
```

3.3 ★★ StringBuilder, speed and the string pool

1. Write `BuilderTools` with: `isPalindrome(String s)` ignoring case and non-letters (use `StringBuilder.reverse()`); `reverseWords(String s)` (`"Java is  fun"` → `"fun is Java"`, single spaces in the result); `compress(String s)`, a run-length encoding (`"aaabccdddd"` → `"a3b1c2d4"`); `group(String s, int n, char sep)`, which inserts `sep` every n characters counted from the left, using `insert` on one builder.

```
IO.println(isPalindrome("A man, a plan, a canal: Panama") + " " + isPalindrome("Java"));
IO.println(reverseWords("Java is  fun"));
IO.println(compress("aaabccdddd") + " " + compress("") + "|");
IO.println(group("ABCDEFGH", 3, '-') + " " + group("ABCDEF", 3, '-') + " " + group("AB", 3, '-'));
IO.println(new StringBuilder("abcdef").delete(1, 3).insert(0, "<").append('>'));
```

Sample run:

```
true false
fun is Java
a3b1c2d4 |
ABC-DEF-GH ABC-DEF AB
<adef>
```

2. Write `ConcatBenchmark`: build the string of the numbers 0 … N−1 (N = 100 000) three ways – `String +=`, `StringBuffer`, `StringBuilder` – time each with `System.currentTimeMillis()`, and check that the three results are equal. Your times will differ; the order of magnitude is what matters.

```
String        4094 ms
StringBuffer  13 ms
StringBuilder 8 ms
same result: true, length 488890
```

3. Write `PoolDemo` with `countSameObject(String[] words, String target)` (counts with `==`), `countEqual(String[] words, String target)` (counts with `equals`) and `internAll(String[] words)` (replaces every element with its `intern()`), and run:

```
String half = "ja";
String[] words = { "java", new String("java"), half + "va", new String(new char[]{'j', 'a', 'v',
↪  'a'}), "python" };
IO.println(countSameObject(words, "java") + " " + countEqual(words, "java"));
internAll(words);
IO.println(countSameObject(words, "java") + " " + countEqual(words, "java"));
```

Sample run:

```
1 4
4 4
```

3.4 ★★★ Encoding tools

Write `EncodingTools` (with `java.nio.charset.Charset`):

- `hex(byte[] bytes)`: `"50 65 72"`-style uppercase hex, two digits per byte (bytes are signed);
- `dump(String s, String charsetName)`: `Perché in UTF-8: 50 65 72 63 68 C3 A9 (7 bytes)`;
- `breakIt(String s)`: simulates the classic bug – the UTF-8 bytes of `s` decoded as ISO-8859-1;
- `fixMojibake(String broken)`: repairs it (get the ISO-8859-1 bytes back, decode them as UTF-8);
- `codePointLength(String s)`: the number of code points, not of chars (`s.codePointCount(0, s.length())`);
- `canEncode(String s, String charsetName)`: uses the charset's encoder (`newEncoder().canEncode(s)`).

Run it with `java -Dstdout.encoding=UTF-8 ...` so the console can show the accented characters.

```
IO.println(dump("Perché", "UTF-8"));
IO.println(dump("Perché", "ISO-8859-1"));
IO.println(dump("5€", "UTF-16BE"));
String broken = breakIt("Perché costa 5€?");
IO.println(broken.equals("Perché costa 5€?") + " -> " + fixMojibake(broken));
IO.println(fixMojibake("PerchÃ©"));
String smile = "😀 ok";
IO.println(smile.length() + " chars, " + codePointLength(smile) + " code points");
IO.println(canEncode("Perché", "ISO-8859-1") + " " + canEncode("5€", "ISO-8859-1") + " " +
↪  canEncode("5€", "UTF-8"));
IO.println(new String("5€".getBytes(Charset.forName("ISO-8859-1")),
↪  Charset.forName("ISO-8859-1")));
```

Sample run:

```
Perché in UTF-8: 50 65 72 63 68 C3 A9 (7 bytes)
Perché in ISO-8859-1: 50 65 72 63 68 E9 (6 bytes)
5€ in UTF-16BE: 00 35 20 AC (4 bytes)
false -> Perché costa 5€?
Perché
5 chars, 4 code points
true false true
5?
```

Part 4 – JUnit 5 (J03)

4.1 ★ Test the slides' Stack

Take the `Stack` class from the slides (add the missing `;` after `throw new StackException()`) and `public class StackException extends Exception { }`. Write `StackTest` with JUnit 5:

- a `@BeforeEach` method that creates a stack of capacity 3;

- a new stack is empty (assertion with a message);
- after a push the stack is not empty;
- pushing 1, 2, 3 and popping gives 3, 2, 1, then the stack is empty;
- the 4th push on a full stack returns false and does not store the element;
- `pop()` on an empty stack throws `StackException` (`assertThrows`);
- `new Stack()` holds exactly 10 elements.

Expected: 6 green tests.

4.2 ★★ Test the bank account, then a suite

Write `BankAccountTest` for the `BankAccount` of 2.3:

- `@BeforeEach` creates `new BankAccount("Alice", 100)`;
- tests for: the initial state; an account without initial balance has 0; deposit; withdraw (compare doubles with a tolerance, e.g. `1e-9`); deposits of 0 and -5 are rejected and leave the balance unchanged; no overdraft (100.01); on an account with 5000, withdrawing exactly `MAX_WITHDRAWAL` works but 1000.01 is rejected; a negative initial balance is rejected; two new accounts get consecutive numbers (do not assert absolute numbers: the counter is static and shared by all tests);
- a static `@BeforeAll` that prints `BankAccountTest: start`, an `@AfterEach` that counts the tests in a static counter, and a static `@AfterAll` that prints `BankAccountTest: <n> tests run`.

Then write a suite class `AllTests` that runs `StackTest` and `BankAccountTest` together (`@Suite`, `@SelectClasses`).

Finally write `MiscTest`, which uses the remaining assertions on earlier classes: `assertNotNull(shop.find("pen"))` and `assertNull(shop.find("eraser"))` on a `Shop` of 2.4; `assertSame(Temperature.ofCelsius(0), Temperature.ofFahrenheit(32))` and `assertNotSame` on two `ofCelsius(20)` of 2.7; and a test that calls `fail("sold more than the stock")` if `sell("pen", 100)` on a stock of 20 does not return -1.

Expected, running each class on its own: `BankAccountTest` 9 green, with `BankAccountTest: start` printed before them and `BankAccountTest: 9 tests run` after; `AllTests` 15 green (6 + 9); `MiscTest` 3 green. (If you run everything at once, `BankAccountTest` runs twice – directly and inside the suite – and the static counter goes on to 18.)

4.3 ★★★ Bug reproduction and TDD

1. Bug report: *"`capitalize("")` crashes, and `capitalize("hELLO")` returns `HELLO` instead of `Hello`."* The code is:

```
public static String capitalize(String w) {
    return Character.toUpperCase(w.charAt(0)) + w.substring(1);
}
```

Put it in a class `StringUtils`. Write `CapitalizeTest` with four tests, all using `assertEquals` or `assertThrows`: `capitalize("hELLO")` returns `"Hello"`; `capitalize("")` returns `""`; `capitalize("a")` returns `"A"`; `capitalize(null)` throws `IllegalArgumentException`. Run it on the buggy code and watch it go red: 2 failures (an assertion not met), 1 error (the crash) and 1 pass – the JUnit 5 console counts failures and errors together as "3 tests failed". Then fix `capitalize` until all 4 are green.

2. Test-driven development: `Roman.toRoman(int n)` converts 1–3999 to Roman numerals and throws `IllegalArgumentException` otherwise. Write `RomanTest` **first**: single symbols (1, 5, 10, 1000), additive forms (3, 8, 14), subtractive forms (4, 9, 40, 90, 400, 900), long numbers (1994 → `MCMXCIV`, 2026 → `MMXXVI`, 3999 → `MMMCMXCIX`), out of range (0, 4000, -7). Watch it fail, then implement `toRoman` until it is green. Finally:
   - add a stub `fromRoman(String s)` that throws `UnsupportedOperationException`, and a test for it marked `@Disabled("fromRoman not implemented yet")`;
   - add a slow `exhaustiveCheck` test over 1–3999 (only the symbols `MDCLXVI`, never `IIII`, `XXXX`, `CCCC`) that runs only when the environment variable `SLOW_TESTS` is set (`assumeTrue`).

Expected: 5 green, 1 skipped (disabled) and 1 aborted (assumption); with `SLOW_TESTS=1`: 6 green and 1 skipped.

Part 5 – Inheritance, interfaces, lambdas (J04)

5.1 ★ Employees, managers, CEO

- `Employee`: private `name` and `wage`, constructor, getters, `incrementWage(double percent)` (`wage += wage * percent / 100`), `totalPay()` (the wage), `describe()` → `"<name> earns <wage>"`, and `toString()` returning `describe()`.
- `Manager extends Employee`: a `unit`, constructor `Manager(name, wage, unit)` (calls `super(...)`), `getUnit()`, `changeUnit(String)`, and `describe()` that reuses the parent's and appends `", manages <unit>"`.
- `CEO extends Manager`: a `bonus`; constructor `CEO(name, wage, bonus)` with unit `"the company"`; overrides `totalPay()` (wage + bonus) and `describe()` (appends `", bonus <bonus>"`).
- `Payroll` with static methods taking `Employee... staff`: `totalPay`, `raiseAll(double percent, ...)`, `countManagers` (`instanceof`) and `findManagerOf(String unit, ...)` (pattern-matching `instanceof`; `null` if none).

```
Employee[] staff = {
    new Manager("Mary Black", 25000, "IT"),
    new Employee("John Smith", 12000),
    new CEO("Ada King", 90000, 10000),
    new Employee("Jane Doe", 12000)
};
Payroll.raiseAll(10, staff);
for (Employee e : staff) {
    IO.println(e);
}
IO.println("Total pay: " + Payroll.totalPay(staff));
IO.println("Managers: " + Payroll.countManagers(staff));
Manager m = Payroll.findManagerOf("IT", staff);
m.changeUnit("HR");
IO.println(staff[0]);
IO.println(Payroll.findManagerOf("Sales", staff));
((Manager) staff[2]).changeUnit("Board");
IO.println(staff[2]);
```

Sample run:

```
Mary Black earns 27500.0, manages IT
John Smith earns 13200.0
Ada King earns 99000.0, manages the company, bonus 10000.0
Jane Doe earns 13200.0
Total pay: 162900.0
Managers: 2
Mary Black earns 27500.0, manages HR
null
Ada King earns 99000.0, manages Board, bonus 10000.0
```

5.2 ★★ equals, hashCode, toString, clone

1. `Student(int id, String name)`: two students are equal when they have the same id. Override `toString()` (`Student[id=7, name=Ada]`), `equals(Object)` respecting the contract (also `equals(null)` is false, and a non-Student is never equal) and `hashCode()` consistent with it.
2. `static int countDistinct(Object... items)`: how many different objects according to `equals` (no collections).
3. `Polygon` with `String name` and `int[] xs` (constructor `Polygon(String name, int... xs)`), implementing `Cloneable` with `public Polygon clone() throws CloneNotSupportedException` based on `super.clone()`. Changing the copy's array must not change the original. `toString()` → `tri[1, 2, 3]`.

```
// main declared with: throws CloneNotSupportedException
Student a = new Student(7, "Ada");
Student b = new Student(7, "Ada L.");
Student c = new Student(8, "Bob");
IO.println(a + " " + a.equals(b) + " " + a.equals(c) + " " + a.equals(null) + " " +
↪  a.equals("Ada"));
IO.println(a.hashCode() == b.hashCode());
IO.println(countDistinct(a, b, c, "Ada", new Student(8, "Robert")));
Polygon p = new Polygon("tri", 1, 2, 3);
Polygon q = p.clone();
q.xs[0] = 99;
q.name = "copy";
IO.println(p + " " + q);
```

Sample run:

```
Student[id=7, name=Ada] true false false false
true
3
tri[1, 2, 3] copy[99, 2, 3]
```

5.3 ★★ Abstract classes and Template Method

1. Abstract `Shape` with a `name` (protected constructor), `getName()`, abstract `area()` and `perimeter()`, and a **final** template method `describe()` → `Circle: area=3.14 perimeter=6.28` (`String.format(Locale.US, ...)`). Concrete `Circle(r)`, `Rectangle(w, h)` and `Square(side) extends Rectangle` (Square adds no methods: it passes its name and sides to a protected `Rectangle` constructor).
2. Write the abstract `Sorter` of the slides: `sort(Object[] v)` is their bubble sort and calls `protected abstract int compare(Object a, Object b)` (the slide declares `compare` package-private and has a stray `}`, so it does not compile as copied). Then write `StringSorter`,

`LengthSorter` (shorter strings first) and `AreaSorter` (shapes by area); their `compare` must be `protected` or `public`.

```
Shape[] shapes = { new Rectangle(3, 4), new Circle(1), new Square(2) };
for (Shape s : shapes) {
    IO.println(s.describe());
}
new AreaSorter().sort(shapes);
String order = "";
for (Shape s : shapes) {
    order += s.getName() + " ";
}
IO.println("by area: " + order.trim());
String[] v = {"g", "t", "h", "n", "j", "k"};
new StringSorter().sort(v);
IO.println(Arrays.toString(v));
String[] w = {"pear", "fig", "banana", "kiwi"};
Sorter s = new LengthSorter();
s.sort(w);
IO.println(Arrays.toString(w));
```

Sample run:

```
Rectangle: area=12.00 perimeter=14.00
Circle: area=3.14 perimeter=6.28
Square: area=4.00 perimeter=8.00
by area: Circle Square Rectangle
[g, h, j, k, n, t]
[fig, pear, kiwi, banana]
```

5.4 ★★ Interfaces: complex numbers, default methods

1. `interface Complex` with `real()`, `im()`, `mod()`, `arg()`; static factory methods `fromRect(re, im)` and `fromPolar(mod, arg)`; default methods `isReal()` (`im() == 0`), `plus(Complex)`, `times(Complex)` (multiply the moduli, add the arguments) and `asString()` → `3.00+4.00i` (`"%.2f%+.2fi"`, `Locale.US`). Implement it with two package-private classes, `ComplexRect` (stores re, im) and `ComplexPolar` (stores mod, arg).
2. Given `interface Walker { default String move() { return "walk"; } }` and `interface Swimmer { default String move() { return "swim"; } }`, write `interface Amphibian extends Walker, Swimmer` that resolves the clash between the two defaults with its own default `move()` returning `"walk and swim"` (calling both inherited defaults), and `class Duck implements Amphibian` with an empty body.

```
Complex a = Complex.fromRect(3, 4);
Complex b = Complex.fromPolar(2, Math.PI / 2);
IO.println(a.asString() + " mod=" + a.mod());
IO.println(b.asString() + " real? " + b.isReal());
IO.println(a.plus(b).asString());
IO.println(a.times(b).asString());
IO.println(Complex.fromPolar(5, 0).isReal() + " " + Complex.fromRect(-1, 0).isReal());
Complex i = new Complex() {          // an anonymous class: the imaginary unit
    public double real() { return 0; }
    public double im() { return 1; }
    public double mod() { return 1; }
    public double arg() { return Math.PI / 2; }
};
IO.println(i.times(i).asString());
IO.println(new Duck().move());
```

Sample run (`b` is 2i, so it is not real; its real part `2 * cos(PI/2)` is a tiny non-zero number that prints as 0.00. Likewise `i.times(i)` has an imaginary part of about 1.2e-16: it prints `+0.00i`, yet its `isReal()` would be false):

```
3.00+4.00i mod=5.0
0.00+2.00i real? false
3.00+6.00i
-8.00+6.00i
true true
-1.00+0.00i
walk and swim
```

5.5 ★★★ Composite expression tree

Build `interface Expression { double eval(); String formula(); }` with a leaf `Value` and two composites, `Operation` (`+ - * /`) and `Negate` (unary minus). Mark the composites with an empty **flag interface** `Composed`: `formula()` puts parentheses only around sub-expressions that are `Composed`. Put the helper that adds the parentheses in `Expression` as a static method (`Composed` stays empty). `Operation`'s constructor throws `IllegalArgumentException` for any other operator.

```
static void show(Expression e) {
    IO.println(e.formula() + " = " + e.eval());
}
// in main:
show(new Operation('*', new Value(6), new Operation('+', new Value(4), new Value(3))));
show(new Operation('/', new Operation('-', new Value(10), new Value(4)),
                        new Operation('+', new Value(1), new Value(2))));
show(new Operation('-', new Value(10), new Operation('-', new Value(4), new Value(3))));
show(new Negate(new Operation('+', new Value(1), new Value(2))));
show(new Operation('*', new Value(2), new Negate(new Value(3))));
show(new Negate(new Value(5)));
```

Sample run:

```
6.0*(4.0+3.0) = 42.0
(10.0-4.0)/(1.0+2.0) = 2.0
10.0-(4.0-3.0) = 9.0
-(1.0+2.0) = -3.0
2.0*(-3.0) = -6.0
-5.0 = -5.0
```

5.6 ★★★ Comparable, Comparator, Strategy

The slides' `StudentCmp` does not compile as written (`return a.id - b.id;`). Write it properly, then sort students five ways.

- `Student(int id, String name, double average) implements Comparable<Student>`: natural order by id (use `Integer.compare`, never subtraction); getters; `toString()` → `3:Bruno(29.0)`.
- `ByIdDescending implements Comparator<Student>` (the fixed `StudentCmp`, descending).

```
Student[] sv = { new Student(11, "Carla", 27.5), new Student(3, "Bruno", 29.0), new Student(7,
↪  "Alice", 27.5) };
Arrays.sort(sv);                                // 1 natural order
IO.println(Arrays.toString(sv));
Arrays.sort(sv, new ByIdDescending());          // 2 named strategy class
IO.println(Arrays.toString(sv));
// 3 by name, with an anonymous Comparator class
// 4 by average descending, ties by name, with a lambda
// 5 Comparator.comparing(Student::getName).reversed()
```

Sample run:

```
[3:Bruno(29.0), 7:Alice(27.5), 11:Carla(27.5)]
[11:Carla(27.5), 7:Alice(27.5), 3:Bruno(29.0)]
[7:Alice(27.5), 3:Bruno(29.0), 11:Carla(27.5)]
[3:Bruno(29.0), 7:Alice(27.5), 11:Carla(27.5)]
[11:Carla(27.5), 3:Bruno(29.0), 7:Alice(27.5)]
```

5.7 ★★★ Iterable and Iterator

Write `Range(int from, int to, int step) implements Iterable<Integer>` (`to` excluded; `step` $\leq$ 0 throws `IllegalArgumentException`) so that it works in a for-each. Write its iterator as a private inner class; `next()` throws `NoSuchElementException` when there are no more elements. Then write `AnonRange`, the same class with the iterator as an anonymous class.

```
String out = "";
for (int i : new Range(0, 10, 3)) {
    out += i + " ";
}
IO.println(out.trim());
out = "";
for (int i : new AnonRange(0, 10, 3)) {
    out += i + " ";
}
IO.println(out.trim());
int count = 0;
for (int i : new Range(5, 5, 1)) {
    count++;
}
IO.println(count);
Range r = new Range(1, 4, 1);
Iterator<Integer> it1 = r.iterator();
Iterator<Integer> it2 = r.iterator();
IO.println(it1.next() + " " + it1.next() + " " + it2.next());
// finally: sum new Range(0, 10, 3) with an explicit while (hasNext/next) loop and print it
```

Sample run:

```
0 3 6 9
0 3 6 9
0
1 2 1
18
```

5.8 ★★★ Functional interfaces, lambdas, method references

1. Write, with `java.util.function`: `forEachIf(int[] v, IntPredicate cond, IntConsumer action)`, `reduce(int[] v, int identity, IntBinaryOperator op)`, `generate(int n, IntSupplier s)` returning an `int[]`, and `mapToStrings(int[] v, IntFunction<String> f)` returning a `String[]`.
2. Run the code below, then replace every lambda marked `// ->` with the equivalent method reference (static, instance method of a given object, instance method of a type, or constructor): the output must not change.

```
int[] v = {5, -2, 8, 3, -7, 10};
forEachIf(v, x -> x > 0, x -> IO.print("[" + x + "]"));
IO.println();
IO.println(reduce(v, 0, (a, b) -> Integer.sum(a, b)));                  // ->
IO.println(reduce(v, Integer.MIN_VALUE, (a, b) -> Math.max(a, b)));     // ->
IO.println(reduce(v, 1, (a, b) -> a * b));
int[] counter = {0};
IO.println(Arrays.toString(generate(4, () -> ++counter[0] * 10)));
IO.println(Arrays.toString(mapToStrings(new int[]{10, 255}, n -> Integer.toHexString(n))));   // ->

String hexDigits = "0123456789ABCDEF";
IntFunction<Character> hex = i -> hexDigits.charAt(i);                          // ->
Function<String, Integer> len = s -> s.length();                                // ->
Supplier<StringBuilder> maker = () -> new StringBuilder();                      // ->
BiFunction<String, String, Boolean> same = (a, b) -> a.equalsIgnoreCase(b);     // ->
Predicate<String> empty = s -> s.isEmpty();                                     // ->
Consumer<Object> printer = o -> IO.println(o);                                  // ->
IO.println(hex.apply(11));
IO.println(len.apply("lambda"));
IO.println(maker.get().append("new!").reverse());
IO.println(same.apply("Java", "JAVA") + " " + empty.test(""));
printer.accept("Hello!");
```

3. Write the slides' printing `Consumer<Object>` three ways – anonymous class, lambda, method reference – and call them with `"A"`, `"B"`, `"C"`.

Sample run:

```
[5][8][3][10]
17
10
16800
[10, 20, 30, 40]
[a, ff]
B
6
!wen
true true
Hello!
A
B
C
```

5.9 ★★★ Observer by composition (code + test)

1. `Thermostat` holds a `double temperature` and uses a `java.beans.PropertyChangeSupport` by **composition**, as in the slides: `setTemperature(double)` fires a `"temperature"` change, and `addListener` / `removeListener` delegate to the support object. Write `Display implements PropertyChangeListener` printing `Display: <old> -> <new>`, and an alarm written as a **lambda** that prints `ALARM: <t>` when t > 30.

```
Thermostat t = new Thermostat(20.0);
Display d = new Display();
t.addListener(d);
t.addListener(/* the alarm lambda */);
t.setTemperature(22.5);
t.setTemperature(22.5);
t.setTemperature(31.0);
t.removeListener(d);
t.setTemperature(35.0);
```

Sample run (setting the same value twice notifies once):

```
Display: 20.0 -> 22.5
Display: 22.5 -> 31.0
ALARM: 31.0
ALARM: 35.0
```

2. Write `ThermostatTest` with a recording listener (a lambda that counts the events and stores the last new value in fields of the test class): a change is notified once with the right value; the same value set twice notifies once; a removed listener receives nothing. Expected: 3 green tests.